INS Lab Exp-2

Aim:

Perform encryption and decryption using following Transposition techniques
i) Rail fence
ii) Row Transformation
iii) Column Transformation.

1. Rail Fence Cipher

Description

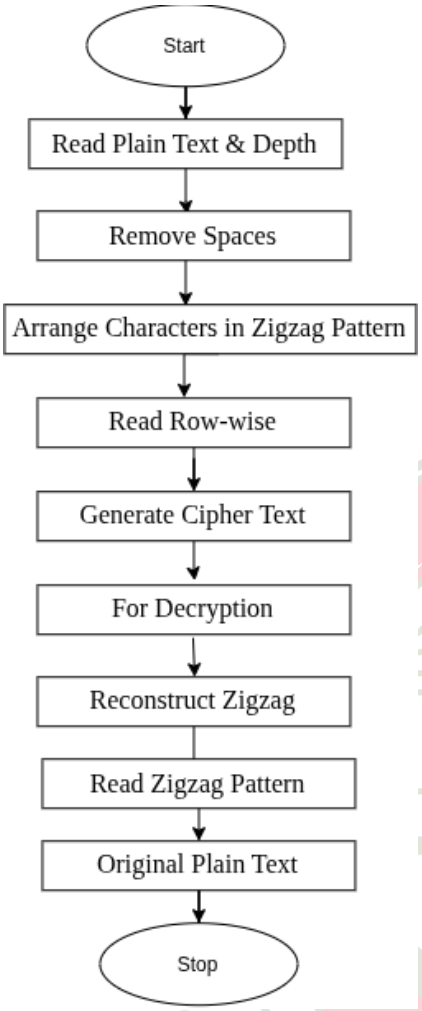
The Rail Fence Cipher is a classical transposition cipher that encrypts a message by arranging its characters in a zigzag pattern across a specified number of rails. After placing all characters, the ciphertext is obtained by reading the characters row by row. Since only the positions of the characters are changed and not the characters themselves, it is known as a transposition cipher.
The decryption process reconstructs the zigzag pattern and retrieves the original plaintext.
Working
Encryption

1. Remove spaces from the plaintext.
2. Create the required number of rails (rows).
3. Write the plaintext characters in a zigzag pattern.
4. Read the rails row by row.
5. The obtained text is the ciphertext.

Decryption

1. Mark the zigzag positions using the given rail depth.
2. Fill the marked positions with ciphertext characters.
3. Traverse the zigzag pattern.
4. Read the characters in zigzag order.
5. Recover the original plaintext.

Flow Chart



Implementation

```
def encrypt(text, depth):
    text = text.replace(" ", "")      # Ignore spaces
    rails = [""] * depth
    row = 0
    down = True
    for ch in text:
        rails[row] += ch
        if row == 0:
            down = True
        elif row == depth - 1:
            down = False
        row = row + 1 if down else row - 1
    return "".join(rails)
def decrypt(cipher, depth):
    n = len(cipher)
    pattern = [[""] * n for _ in range(depth)]
    row = 0
```

```
    down = True
    for col in range(n):
        pattern[row][col] = "*"
        if row == 0:
            down = True
        elif row == depth - 1:
            down = False
        row = row + 1 if down else row - 1
    index = 0
    for i in range(depth):
        for j in range(n):
            if pattern[i][j] == "*":
                pattern[i][j] = cipher[index]
                index += 1
    result = ""
    row = 0
    down = True
    for col in range(n):
        result += pattern[row][col]
        if row == 0:
            down = True
        elif row == depth - 1:
            down = False
        row = row + 1 if down else row - 1
    return result
text = input("Enter Plain Text: ")
depth = int(input("Enter Depth: "))
encrypted = encrypt(text, depth)
decrypted = decrypt(encrypted, depth)
print("Encrypted Text:", encrypted)
print("Decrypted Text:", decrypted)
```

Output

```
Enter Plain Text: INFORMATIONSECURITY
Enter Depth: 3
Encrypted Text: IRIEINOMTOSCRTFANUY
Decrypted Text: INFORMATIONSECURITY
```

Result

The Rail Fence Cipher program successfully encrypted the plaintext into ciphertext and correctly decrypted it back to the original plaintext.

Conclusion

The Rail Fence Cipher successfully demonstrates the concept of transposition encryption by rearranging the positions of characters in a zigzag pattern without changing the characters

themselves. The implementation correctly performs both encryption and decryption using the specified rail depth.

2. Row Transformation Cipher

Description

The Row Transformation Cipher is a transposition cipher in which the plaintext is written row-wise into a matrix. The number of columns is equal to the length of the numeric key provided by the user. The ciphertext is generated by reading the columns directly in the order specified by the digits of the key.

For example,

Key = 3142

The columns are read as:

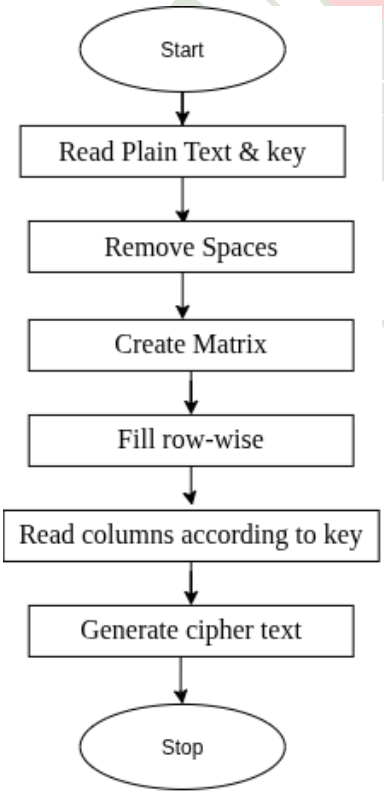
- Column 3
- Column 1
- Column 4
- Column 2

This produces the encrypted message.

Working

1. Remove spaces from the plaintext.
2. Determine the number of columns from the key length.
3. Arrange the plaintext row-wise into a matrix.
4. Read the columns according to the sequence of digits in the key.
5. Concatenate the column contents to generate the ciphertext.

Flow Chart



Implementation

```
import math
def encrypt(text, key):
    text = text.replace(" ", "")      # Remove spaces
    cols = len(key)
    rows = math.ceil(len(text) / cols)
    matrix = [["" for _ in range(cols)] for _ in range(rows)]
    index = 0
    for i in range(rows):
        for j in range(cols):
            if index < len(text):
                matrix[i][j] = text[index]
                index += 1
    cipher = ""
    for num in range(1, cols + 1):
        col = key.index(str(num))     # Find original column having this number
        for i in range(rows):
            if matrix[i][col] != "":
                cipher += matrix[i][col]
    return cipher
text = input("Enter Plain Text: ")
key = input("Enter Key: ")
cipher = encrypt(text, key)
print("Encrypted Text:", cipher)
```

Output

```
Enter Plain Text: WEAREDISCOVEREDFLEEATONCE
Enter Key: 4312567
Encrypted Text: AOLCRVEEECFNWSDOEEEDRAIET
```

Result

The Row Transformation Cipher program successfully generated the ciphertext by rearranging the matrix columns according to the user-provided numeric key.

Conclusion

The Row Transformation Cipher effectively demonstrates transposition encryption by rearranging the columns according to a user-defined numeric key. Different key values produce different ciphertexts while preserving the original characters.

3. Column Transformation Cipher

Description

The Column Transformation Cipher is another transposition cipher in which the plaintext is written row-wise into a matrix. Unlike the Row Transformation Cipher, the columns are read according to the ascending order of the numbers present in the key.

For example,

Key = 3142

The reading order becomes:

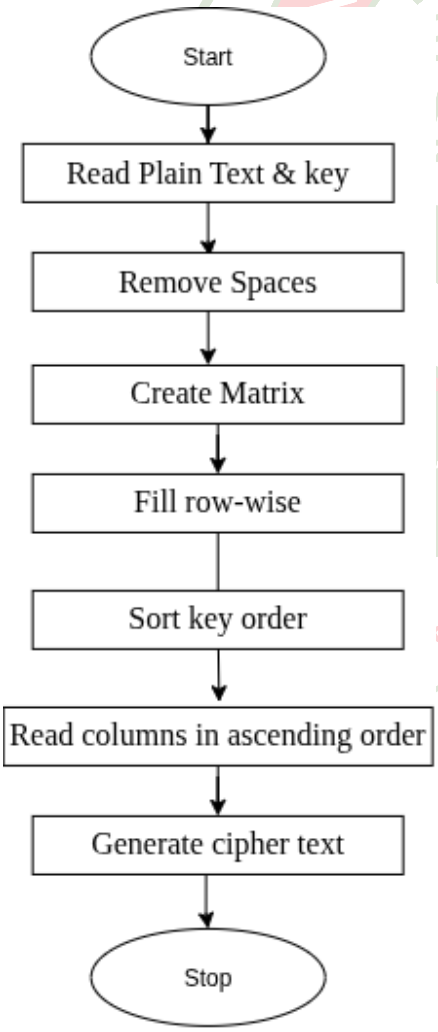
- Number 1 → Column 2
- Number 2 → Column 4
- Number 3 → Column 1
- Number 4 → Column 3

Since the columns are read based on the sorted order of the key values, the generated ciphertext differs from that of the Row Transformation Cipher.

Working

1. Remove spaces from the plaintext.
2. Arrange the plaintext row-wise into a matrix.
3. Determine the position of each key number in ascending order.
4. Read the corresponding columns sequentially.
5. Combine the column contents to obtain the ciphertext.

Flow Chart



Implementation

```
import math
def encrypt(text, key):
    text = text.replace(" ", "")  # Ignore spaces
```

```
    cols = len(key)
    rows = math.ceil(len(text) / cols)
    matrix = [["" for _ in range(cols)] for _ in range(rows)]
    index = 0
    for i in range(rows):
        for j in range(cols):
            if index < len(text):
                matrix[i][j] = text[index]
                index += 1
    cipher = ""
    for digit in key:
        col = int(digit) - 1
        for i in range(rows):
            if matrix[i][col] != "":
                cipher += matrix[i][col]
    return cipher
text = input("Enter Plain Text: ")
key = input("Enter Key: ")
cipher = encrypt(text, key)
print("Encrypted Text:", cipher)
```

Output

```
Enter Plain Text: ATTACKATDAWN
Enter Key: 3142
Encrypted Text: TAWACDATNTKA
```

Result

The Column Transformation Cipher program successfully encrypted the plaintext by reading the matrix columns according to the ascending order of the numeric key.

Conclusion

The Column Transformation Cipher demonstrates column permutation using a numeric key. By reading the matrix columns in ascending key order, it produces a ciphertext different from the Row Transformation Cipher while preserving the original characters. This illustrates the effectiveness of key-based transposition in classical cryptography.